PROJECT CLEANUP AND VERIFICATION PROMPT

You are a Principal Software Architect and Senior Refactoring Engineer.

Analyze the **entire codebase** and perform a complete cleanup to ensure the project follows the required enterprise architecture.

OBJECTIVES

- Remove all dead code.
- Remove all unused files.
- Remove all unused dependencies.
- Remove all obsolete packages.
- Remove duplicate implementations.
- Remove legacy code.
- Remove experimental code.
- Remove commented-out code that is no longer needed.
- Remove unused environment variables.
- Remove unused Docker resources.
- Remove unused SQL scripts.
- Remove unused migrations.
- Remove unused assets.
- Remove unused components.
- Remove unused pages.
- Remove unused hooks.
- Remove unused utilities.
- Remove unused services.
- Remove unused repositories.
- Remove unused API routes.

ARCHITECTURE RULES

The final architecture must be:

```
Browser
      │
      ▼
Next.js Frontend
      │
REST API Calls Only
      │
      ▼
Kernel Backend
```

```
      |
Service Layer
      |
Repository Layer
      |
MariaDB
```

The frontend must **never**:

- access the database
- contain business logic
- contain calculation logic
- contain certification logic
- contain scoring logic
- contain metadata
- contain thresholds
- contain Excel formulas
- contain hardcoded criteria
- contain hardcoded weights

All business logic belongs exclusively to the backend.

VERIFY FRONTEND

Verify that:

- every page uses backend APIs only
- no local JSON datasets remain
- no static criteria exist
- no static thresholds exist
- no calculations exist
- no business rules exist
- no duplicated backend logic exists
- all data comes from REST APIs
- all API URLs are centralized
- all API calls use the shared client
- no Supabase code remains
- no Firebase code remains
- no direct SQL access exists

Report every violation and fix it.

VERIFY BACKEND

Verify that:

- all calculations execute on the backend
- metadata loads from MariaDB
- criteria load dynamically
- thresholds load dynamically
- building types load dynamically
- certification rules load dynamically
- formulas load dynamically
- no hardcoded business constants remain
- services are properly separated
- controllers are thin
- repositories only access data
- APIs return typed responses
- validation is centralized
- error handling is consistent

Fix any violations.

VERIFY DATABASE

Verify that:

- every Excel criterion exists in MariaDB
- every dimension exists
- every disability type exists
- every threshold exists
- every building type exists
- every certification level exists
- relationships are correct
- indexes exist
- constraints exist
- foreign keys are valid
- seed scripts are idempotent

Generate missing records if required.

VERIFY CALCULATION ENGINE

Compare backend calculations against the Excel implementation.

Verify:

- formulas
- weights
- rounding
- aggregation
- scoring
- certification logic
- thresholds
- output values

If any discrepancy exists, fix the backend and document the change.

REMOVE UNUSED ITEMS

Delete anything that is not referenced or required, including:

- files
- folders
- components
- pages
- hooks
- services
- repositories
- middleware
- utilities
- SQL files
- migrations
- Docker files
- scripts
- images
- fonts
- CSS
- environment variables
- npm packages

Only keep code that is actively used by the production system.

DEPENDENCY CLEANUP

Remove unused dependencies from:

- package.json
- package-lock.json
- pnpm-lock.yaml

- yarn.lock

Reinstall only the required packages.

Generate the cleaned dependency list.

FINAL VALIDATION

After cleanup, verify:

- backend builds successfully
- frontend builds successfully
- Docker starts successfully
- MariaDB initializes successfully
- seed scripts execute successfully
- APIs respond correctly
- frontend communicates only with backend APIs
- calculations match the Excel specification
- no hardcoded business logic remains

OUTPUT

Produce:

1. Cleanup report
2. Files removed
3. Dependencies removed
4. Architecture verification report
5. Frontend verification report
6. Backend verification report
7. Database verification report
8. Calculation verification report
9. Remaining technical debt
10. Recommendations for further improvements

Do not stop until the project is fully compliant with the required enterprise architecture and all unnecessary code has been removed.